# QWeave: Correctness-First Deterministic and Learned Qubit Routing

Atharva Sheersh Pandey*, Diptesh Das*, Shrivardhini N*, Haridasu Sreedhar*, Prof. Bhuvaneswari M†
*School of Computer Science and Engineering, VIT Vellore
†Faculty Guide, School of Computer Science and Engineering, VIT Vellore
{atharva.sheersh2024, diptesh.das2024, shrivardhini.n2024, haridasu.sreedhar2024}@vitstudent.ac.in

***Abstract*—Restricted two-qubit connectivity requires a compiler to place logical qubits on physical sites and insert routing operations before nonlocal interactions. QWeave is a correctness-first quantum compiler combining weighted interaction-graph placement, strict local improvement, deterministic shortest-path routing, fixed-route scheduling, directed-gate lowering, and a guarded graph-neural-network/proximal-policy-optimization (GNN–PPO) router. Formal arguments establish mapping-search termination, routing-layout invariants, supported unitary semantics, fixed-DAG scheduling optimality, directed-CNOT equivalence, and guarded-route termination. We report two *separate* experimental studies. Frozen benchmark v2 comprises 180 synthetic cases, including 36 held-out test cases and 792 evaluated outputs across five training seeds; all outputs passed applicable semantic checks, and all direction-lowered cost circuits passed ordered-coupler legality. The paired GNN–PPO minus weighted depth difference was 0 gates (95% bootstrap interval [0,0]), with no demonstrated learned-routing advantage. A supplementary 30-run circuit–topology study compares deterministic Weighted, Basic, and Qiskit SABRE routing: Weighted inserts 59 SWAPs versus 77 and 62, respectively, and has total routed depth 297 versus 306 and 300. However, its advantage concentrates in hub-structured workloads and reverses on several sparse-topology, dense-interaction cases. These studies support correctness and workload-dependent placement benefits, not universal superiority or measured hardware fidelity.**

***Index Terms*—quantum compilation, qubit mapping, qubit routing, circuit depth, reinforcement learning, graph neural networks, correctness, SABRE**

## I. Introduction

Quantum algorithms are generally expressed with unrestricted logical two-qubit interactions, whereas near-term devices expose a finite coupling graph. Compilation must assign logical qubits to physical sites, insert operations that resolve nonlocal interactions, and preserve the logical computation. Qiskit distinguishes layout, routing, translation, optimization, and scheduling stages [1]; SABRE uses bidirectional heuristic search for mapping and SWAP selection [2] and is therefore a substantive baseline.

QWeave studies a narrower and reproducible question: under a declared circuit language, topology, mapping convention, and validation protocol, what do deterministic placement, guarded learned routing, and fixed-route scheduling contribute? Edge legality alone is insufficient: a compiler can use legal couplers while implementing the wrong operation because of incorrect layout updates, SWAP handling, parameters, or classical destinations. Accordingly, QWeave couples trace replay with numerical checks and reports placement, routing, scheduling, and direction lowering separately.

The contributions are: (i) a deterministic mapping and routing reference with an explicit adapter contract; (ii) formal preservation, feasibility, and termination arguments; (iii) a guarded edge-conditioned GNN–PPO routing implementation with meaningful no-message and untrained controls; (iv) frozen held-out benchmark-v2 evaluation with paired uncertainty estimates; and (v) an *additional, separately analyzed* 30-run study of circuit–topology interactions that explains where weighted placement helps and where it fails. Neither graph encoding nor PPO is presented as inherently novel, and no learned-policy superiority is claimed.

## II. Related Work

The NISQ compilation setting and retargetable compiler stacks are established by quantum-computing and compiler literature [5], [6], [7], [8], [9], [10], [11]. Qubit allocation, architecture-aware mapping, and circuit routing have been treated as related but distinct optimization problems [12], [13], [14], [15]. Look-ahead search, exact SWAP minimization, subgraph placement, commutation-aware transformations, gate absorption, and topology-aware assignment demonstrate that routing performance depends on circuit structure and available rewrites [16], [17], [18], [19], [20], [21], [22].

Noise-adaptive and variability-aware compilation, timing-aware remapping, and hardware-specific optimization show why SWAP count, circuit depth, duration, and physical fidelity must not be conflated [23], [24], [25], [26], [27], [28], [29], [55], [56], [57], [58], [59], [60]. In particular, correlated errors and crosstalk invalidate a simplistic independent-error model [61], [62], [63], [64], [65], [66], [67]. QWeave's declared error profile is therefore only a synthetic success-probability proxy.

Reinforcement learning for routing and learned layout selection precede this work [3], [4], [34], [35]; GNN message passing and PPO are established methods [36], [37], [38], [39], [73], [74], [75]. The present learned-routing study evaluates these ingredients through controlled ablations, following reproducibility guidance for deep RL [71], [72]. Exact optimization and planning can provide reference solutions [40], [41], [42], but QWeave's small CP-SAT oracle solves only an initial-layout surrogate. Interaction-graph and application-driven benchmark work motivates explicit circuit-family and

topology stratification [43], [44], [45], [46], [47], [48], [68], [69], [70].

## III. Problem Formulation and Deterministic Path

Let $C = (g_1, \ldots, g_T)$ be an ordered supported circuit on logical qubits $L = \{0, \ldots, n-1\}$, and let $H = (P, E_H)$ be a finite, connected, undirected routing graph. An injective layout $m : L \to P$ follows the convention $m(\text{logical}) = \text{physical}$. The interaction graph $G_C$ includes isolated logical qubits; its edge weight $w_{ij}$ counts two-qubit interactions. The placement objective is

$$J(m) = \sum_{(i,j)\in E_C} w_{ij}\, d_H\big(m(i), m(j)\big), \tag{1}$$

where $d_H$ is hardware hop distance. An optional temporal weighting exists, but the frozen benchmark uses integer interaction counts.

The weighted mapper seeds a high weighted-degree logical qubit at a high-centrality physical site, greedily assigns remaining qubits using interaction weights and partial distance cost, and applies best-improving pairwise exchange. Exchanges are accepted only if they strictly reduce $J$. The Basic control uses identity placement with the *same* deterministic router, isolating the placement contribution. SABRE is an independent Qiskit reference with fixed seeds and a consistent undirected-adjacency interpretation.

For each source operation, the deterministic router emits a supported gate if its operands are adjacent; otherwise it selects a deterministic shortest path and moves the first logical qubit toward the second by adjacent SWAPs. It updates both forward and inverse layouts after every SWAP. The Qiskit adapter accepts unconditioned one- and two-qubit operations, single-qubit resets, and a terminal measurement suffix, preserving global phase and classical destinations. Barriers, higher-arity operations, dynamic control, and conditioned gates are rejected rather than silently transformed. A replayable trace records source gates and inserted SWAPs.

## IV. Formal Correctness Arguments

The statements below hold under the declared contract: $|L| \le |P|$, connected finite simple routing graph, injective layout, supported gates, positive declared durations, and explicit seeds. Directed couplers are handled by a subsequent lowering pass; unsupported dynamic control is outside scope.

**Proposition 1** (Mapping-search termination)**.** *Strict pair-exchange local search terminates after finitely many accepted exchanges.*

*Proof:* The number of injective layouts is finite:

$$N_{\text{map}} = \frac{|P|!}{(|P| - |L|)!}. \tag{2}$$

Each accepted exchange strictly reduces $J$, so a previously visited layout cannot recur. At most $N_{\text{map}} - 1$ exchanges are accepted. This proves termination, not global optimality. ∎

**Proposition 2** (Layout and legality invariant)**.** *After each emitted operation, the forward layout remains injective, its inverse agrees on occupied physical sites, and every routed two-qubit operation uses an edge in $E_H$.*

*Proof:* A SWAP transposes the states at adjacent physical sites and updates both maps consistently, preserving injectivity and inverse agreement. One-qubit gates cannot violate edge legality; two-qubit source gates are emitted only when adjacent, either immediately or after legal shortest-path SWAPs. Induction on emitted operations proves the claim. ∎

**Lemma 1** (Masked-policy legality)**.** *For state $s$, nonempty legal action set $A_{\text{legal}}(s)$, and logits $z_\theta(s, a)$, the masked policy*

$$\pi_\theta(a \mid s) = \begin{cases} \dfrac{e^{z_\theta(s,a)}}{\sum_{b\in A_{\text{legal}}(s)} e^{z_\theta(s,b)}} & a \in A_{\text{legal}}(s), \\ 0 & \textit{otherwise} \end{cases} \tag{3}$$

*assigns zero probability to illegal actions and unit total probability to legal actions.*

*Proof:* The denominator is positive; summing the legal-action probabilities gives one by construction. ∎

**Theorem 1** (Supported unitary semantics)**.** *For an equal-width supported unitary circuit, the routed prefix $R_k$, logical prefix $C_k$, and layout $m_k$ satisfy*

$$U_{R_k} P_{m_0} = P_{m_k} U_{C_k}, \qquad 0 \le k \le T. \tag{4}$$

*Proof:* At $k = 0$ both prefixes are identities. For adjacent physical transposition $\tau = (u\ v)$, its SWAP obeys $S_{uv} P_m = P_{\tau\circ m}$. The physical realization of the next logical gate, once operands are adjacent, obeys $\widetilde{g}_{k+1} P_m = P_m g_{k+1}$. Applying these identities inductively proves (4). The final unitary is equal up to the declared input and output wire permutations and global-phase convention. ∎

**Proposition 3** (Fixed-route scheduling)**.** *For positive gate durations and only per-qubit exclusivity constraints, earliest-start list scheduling is feasible and minimizes makespan among schedules preserving the fixed per-qubit precedence DAG.*

*Proof:* Let $\text{pred}(i)$ be the DAG predecessors of operation $i$. The recurrence

$$s_i = \max_{j\in\text{pred}(i)} (s_j + d_j) \tag{5}$$

with empty maximum zero starts each operation after its predecessors. The resulting makespan equals the weighted longest-path bound of the DAG; every feasible schedule respecting the same DAG has at least that makespan. The claim does not allow commutation or additional control-resource constraints. ∎

**Proposition 4** (Directed lowering)**.** *If an undirected hardware edge supports CNOT in at least one direction, either requested orientation and an edge-local SWAP can be implemented using available directed CNOTs and single-qubit Hadamards.*

*Proof:* The reverse direction obeys

$$\mathrm{CX}_{u\to v} = (H_u \otimes H_v)\,\mathrm{CX}_{v\to u}\,(H_u \otimes H_v). \tag{6}$$

A SWAP is three alternating CNOTs; applying the identity to unavailable orientations preserves the unitary and ordered-arc legality. ∎

**Proposition 5** (Guarded learned-route termination)**.** *For a finite supported circuit on a finite connected routing graph, the guarded learned environment terminates with a legal route.*

*Proof:* A finite policy-step budget is imposed per pending gate. On repeated nonprogress or exhaustion, deterministic shortest-path fallback performs at most $|P| - 2$ SWAPs before executing the pending two-qubit gate. For $T$ source gates and at most $B$ policy actions per pending gate, the number of actions is bounded by $T(B + |P| - 1)$. Structural replay validates the returned route. ∎

**Proposition 6** (Independent-error proxy)**.** *For declared independent gate-error probabilities $0 \le \epsilon_g < 1$, define*

$$F_b(R) = \prod_{g\in R} (1 - \epsilon_g). \tag{7}$$

*Appending a gate with nonzero error cannot increase $F_b$; an extra SWAP decomposed into three CNOTs multiplies it by $(1 - \epsilon_2)^3$ before direction-reversal overhead.*

*Proof:* Every factor is at most one, so multiplication cannot increase the product. This is a property of the declared model, not measured physical fidelity. ∎

## V. Learned Router

The Gymnasium environment exposes an *Execute* action and one SWAP action per sorted hardware edge. Its observation contains five features per physical node, four per hardware edge, a five-value frontier description, the logical-to-physical mapping, and a legal-action mask. Invalid actions do not change the layout or circuit. Fallback counts are reported because a safe fallback may hide weak policy behavior.

The actor–critic applies two rounds of edge-conditioned message passing and scores each candidate edge SWAP plus Execute. PPO employs clipped policy ratios, generalized advantage estimation, value and entropy terms, and gradient clipping [38], [39]. A zero-message-passing PPO control retains the optimizer and reward; an untrained GNN retains the architecture without PPO updates. All start from the weighted deterministic layout, so the experiment compares *routing policies*, not learned initial placements.

## VI. Validation and Experimental Protocol

Structural replay checks mapping completeness, injectivity, source-gate order, inserted SWAP consistency, global phase, and edge legality. For equal-width unitary circuits of at most six qubits, the final instance of (4) is checked by operator equality. Wider circuits and outputs with idle physical sites use reproducible layout-aware statevector probes up to 12 physical qubits; a finite probe set is not a proof over all inputs. Reset and terminal-measurement circuits receive structural replay, while dynamic control is unsupported.

A fixed-route scheduler reports layer depth or declared-duration makespan. A separate hardware-cost pass lowers CNOTs and SWAPs to directed couplers, verifies ordered arcs, and computes the synthetic independent-error proxy (7). SWAP count is discrete; its empirical probability mass and cumulative distribution, rather than a continuous probability density, are reported.

### A. Frozen benchmark v2

Benchmark v2 freezes 180 synthetic cases at four, six, and eight logical qubits: 60 training, 84 validation, and 36 untouched test cases. Training families include GHZ, layered-CX, seeded interaction, Bernstein–Vazirani, and QAOA MaxCut [49]; the test set includes QFT [50] and hardware-efficient circuits across two-row grids, chorded rings, and larger grids with idle sites. Each case records source QASM, hash, routing and directed edges, and synthetic error parameters.

Three policy candidates receive 1,024 training steps, with validation seed 101; selection minimizes median depth plus half the SWAP count plus five times fallback count. The selected candidate receives 4,096 training steps at seeds 7, 17, 29, 41, and 53 for both GNN–PPO and no-message PPO. The untrained control uses corresponding initialization seeds, SABRE uses the same five seeds, and Basic/Weighted are deterministic. Runtime is the median of three measurements after warmup. Five-seed results are collapsed within each test case before paired differences. Confidence intervals use 5,000 deterministic bootstrap resamples (seed 20260921).

### B. Separate 30-run supplementary study

The additional study contains 30 single dashboard compilations, *not additional records from frozen benchmark v2*. It compares QWeave Weighted, QWeave Basic, and Qiskit SABRE on five circuit classes: GHZ (4 and 6 qubits), star graph (4 and 5), Bernstein–Vazirani (6), hardware-efficient ansatz (4), and QFT (4). Seven topology types are represented: linear path, ring, chorded ring, $2\times2$ and $2\times3$ grids, star, and the five-qubit IBM Manila T-coupling map (4–8 physical qubits). The three methods are reported side by side for SWAPs, routed depth, and one wall-clock compile-time observation per run. The dashboard showed *Weighted* validation as Passed in the 30 runs; baseline validation was not independently displayed in the supplied evidence. This study uses one fixed SABRE seed, and the tabulated values were transcribed from screenshots, subject to result-file verification.

## VII. Results

### A. Frozen benchmark-v2 held-out results

The immutable benchmark was produced under Python 3.11.15 and Qiskit 2.5.2. Its clean-clone run passed the then-current 69-test suite; a later integrated 104-test suite checks expanded CLI/web code but does not replace the 792-record benchmark. All 792 outputs passed applicable numerical semantic checks, and every direction-lowered cost circuit passed

TABLE I: Frozen benchmark-v2 held-out summary (36 cases; median runtime in seconds).

| Method | Depth | SWAP | Runtime | Fallback |
|---|---|---|---|---|
| Basic | 48.5 | 11.5 | 0.0022 | 0 |
| Weighted | **36.0** | **0.0** | 0.0036 | 0 |
| SABRE | 46.0 | 7.0 | 0.0028 | 0 |
| GNN–PPO | 37.5 | 0.0 | 0.3473 | 4543 |
| No-message PPO | 36.0 | 0.0 | 0.1040 | 742 |
| Untrained GNN | 198.0 | 193.0 | 0.6681 | 8059 |

TABLE II: Frozen benchmark-v2 discrete zero-SWAP mass and synthetic success proxy.

| Method | $\widehat{P}(S=0)$ | Median $F_b$ | $F_b$ IQR |
|---|---|---|---|
| Basic | .222 | .526 | [.294,.704] |
| Weighted | .556 | **.691** | [.343,.787] |
| SABRE | .222 | .569 | [.387,.741] |
| GNN–PPO | .556 | .673 | [.204,.787] |
| No-message PPO | .556 | .673 | [.238,.787] |
| Untrained GNN | .000 | .0026 | $[1.29\times10^{-5}$,.024] |

ordered-arc legality. Table I reports the 36 held-out cases after seed collapse.

The paired GNN–PPO minus Weighted depth difference was 0 gates (95% bootstrap interval [0,0]); against SABRE it was 0 [0,5]. Its paired SWAP difference against Weighted was 0 [0,5]. GNN–PPO and no-message PPO had paired median depth and SWAP differences of 0 [0,0], giving no evidence that message passing helped. Training did improve substantially over an untrained GNN: paired depth difference $-162.5$ $[-184, -106]$ and SWAP difference $-175$ $[-223, -148]$. On QFT, Weighted achieved median depth 71.5 and 16.5 SWAPs, versus GNN–PPO 83.5 and 41, and no-message PPO 90 and 31.5. GNN–PPO improved by eight layers on one six-qubit chorded-ring QFT instance but regressed by 89 layers and 130 SWAPs on an eight-qubit 2-row-grid QFT instance. It invoked 4,543 test fallbacks versus 742 for no-message PPO. Seed 29 was unstable late in training.

The paired GNN–PPO minus Weighted log-proxy difference was 0 $[-0.171,0]$, and GNN–PPO minus no-message PPO was 0 [0,0]. After directed lowering, the paired GNN–PPO minus Weighted synthetic makespan difference was 0 ns [0,1113]. These results do not show a learned advantage. The proxy is conditional on synthetic independent-error assumptions and must not be interpreted as hardware fidelity.

### B. Thirty-run deterministic comparison

Table III summarizes the *separate* 30-run study. Weighted inserted 59 SWAPs, versus 77 for Basic (23.4% reduction) and 62 for SABRE (4.8% reduction). Total routed depth was 297, versus 306 (2.9% lower) and 300 (1.0% lower). The mean source-relative depth overhead was 23.0% versus 33.2% and 26.5%. Weighted yielded zero SWAPs on 13 runs, compared with eight each for Basic and SABRE. It had fewer/equal/more SWAPs than Basic in 15/10/5 runs and than SABRE in 11/12/7 runs; for routed depth, the corresponding comparisons were 12/12/6 and 11/15/4. Thus the aggregate benefit is modest and not uniform.

TABLE III: Supplementary 30-run study (separate from benchmark v2).

| Metric | Weighted | Basic | SABRE |
|---|---|---|---|
| Total SWAPs | **59** | 77 | 62 |
| Mean SWAPs/run | **1.97** | 2.57 | 2.07 |
| Total routed depth | **297** | 306 | 300 |
| Mean depth overhead | **23.0%** | 33.2% | 26.5% |
| Zero-SWAP runs | **13** | 8 | 8 |
| Median compile time (ms) | 18.5 | **12.9** | 29.8 |

Times are single interactive-dashboard observations, not benchmarked speed claims.

TABLE IV: Supplementary 30-run head-to-head counts for Weighted (fewer/equal/more SWAPs; lower/equal/higher depth).

| Comparison | SWAPs | Depth |
|---|---|---|
| Versus Basic | 15 / 10 / 5 | 12 / 12 / 6 |
| Versus SABRE | 11 / 12 / 7 | 11 / 15 / 4 |

### C. Circuit–topology interaction and explanation

For 15 hub-structured cases (Bernstein–Vazirani and star graph), Weighted required 13 SWAPs against 33 for Basic and 25 for SABRE, reductions of 60.6% and 48.0%. It used zero SWAPs in all four BV cases (Basic 11 total, SABRE seven). In contrast, for 15 chain-like or dense cases (GHZ, HEA, QFT), Weighted required 46 SWAPs versus 44 and 37. Figure 4 separates circuit classes; Fig. 5 separates hardware topologies. Because circuit family and topology mix can be confounded, Table VI crosses both factors.

The static objective in (1) is a useful proxy when a high-degree hub interacts with many partners and can be placed centrally. On short-distance topologies with alternative paths, placement can also reduce movement. But $J(m)$ ignores gate order and future interactions. On sparse trees and circuits with near-complete or chain-like interaction patterns, a layout with low static cost can nevertheless route poorly. SABRE's look-ahead then provides an advantage. For example, on HEA with star hardware, SABRE used seven SWAPs versus Weighted's 11; on QFT with a line, SABRE used five versus six. For GHZ on an eight-qubit line and IBM Manila, Basic and SABRE achieved zero SWAPs while Weighted added four and one, respectively. These counterexamples are consistent with a local optimum or a weak placement surrogate; the existing data do not distinguish those mechanisms.

Across the 90 method–run observations in the supplementary study, added SWAP count and added depth correlate at $r = 0.97$; added depth equals added SWAPs in 73 observations and does not exceed added SWAPs in 86. SWAPs can sometimes be placed in otherwise idle layers, explaining why a 23% SWAP reduction versus Basic yields only a 2.9% total-depth reduction. Basic had the lowest measured compilation time in 24 of 30 runs; however, the single dashboard observations (medians 12.9, 18.5, and 29.8 ms for Basic, Weighted, and SABRE) are not a controlled speed benchmark.

### D. Relation between the two studies

The frozen v2 and 30-run suites differ in sample size, workload composition, seeding, metric aggregation, and validation evidence. They are *not pooled* into one denominator,

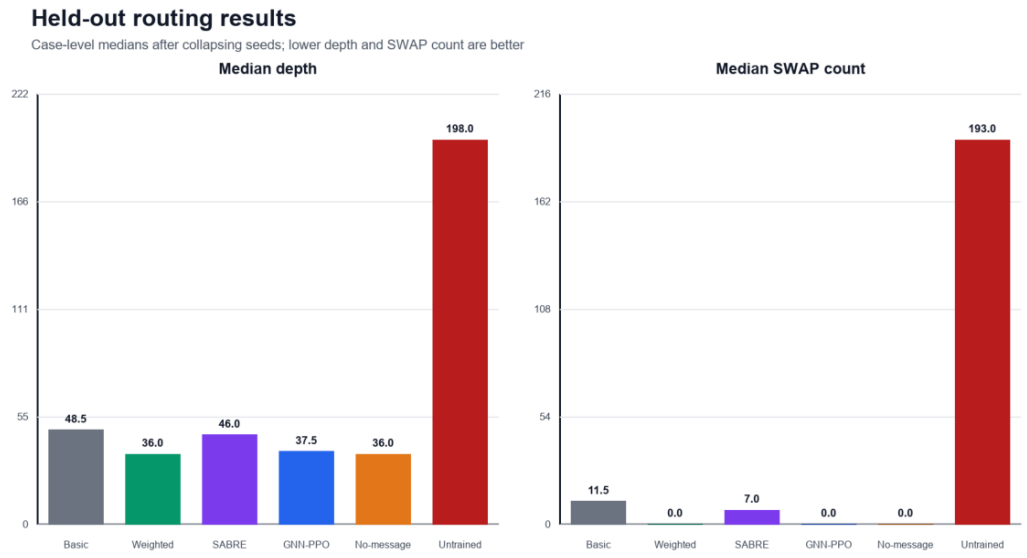



Fig. 1: Frozen benchmark-v2 held-out median routed depth and SWAP count after seed collapse (reproduced from the original QWeave manuscript).

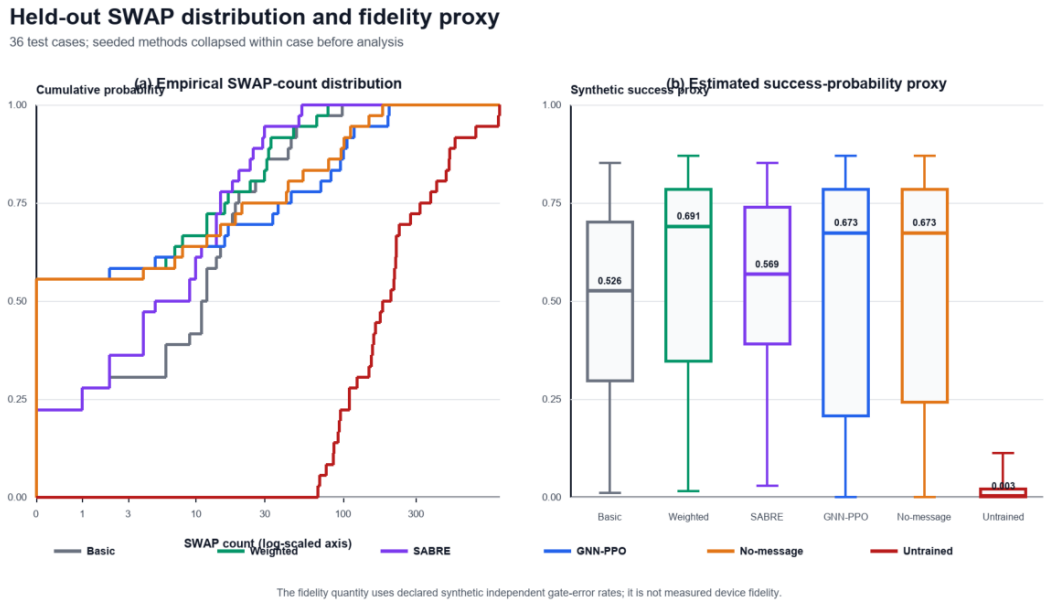



Fig. 2: Frozen benchmark-v2 case-level SWAP cumulative distribution and synthetic success-probability proxy (reproduced from the original QWeave manuscript). The right panel is not measured hardware fidelity.

confidence interval, or performance estimate. The v2 study supports the narrow conclusion that the trained GNN–PPO policy does not improve over Weighted on its held-out test set. The supplementary study shows that Weighted's deterministic placement can outperform Basic and SABRE on selected hub-dominated workloads but can underperform on dense circuits and sparse topologies. These findings are compatible: both identify the weighted deterministic method as a strong reference while exposing its workload dependence.

## VIII. Threats to Validity

The formal guarantees depend on the supported gate contract, connected graph, correct primitive implementations, and

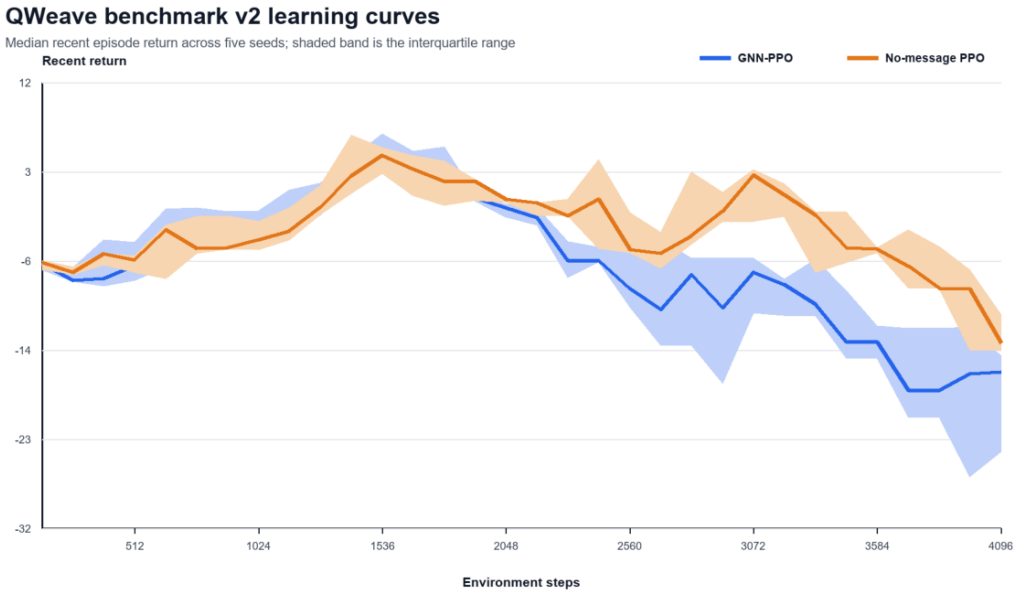



Fig. 3: Frozen benchmark-v2 five-seed training curves: median recent episode return with interquartile bands (reproduced from the original QWeave manuscript). Neither curve is monotonic.

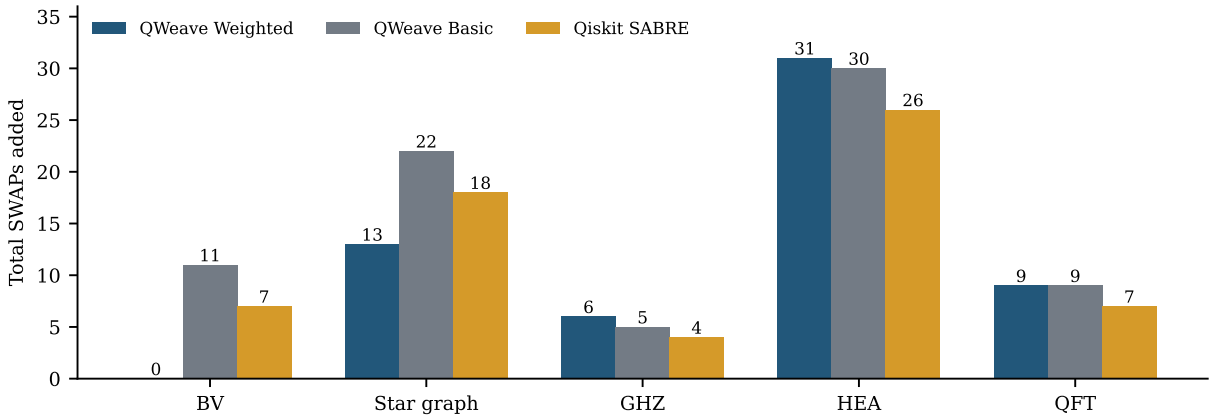



Fig. 4: Supplementary 30-run total SWAPs by circuit class. Weighted has its largest advantage for Bernstein–Vazirani and star-graph circuits, while SABRE is stronger on HEA and QFT. Values are from the supplied per-run table, not benchmark-v2 results.

absence of unmodelled shared resources. Numerical statevector probes for wider circuits are incomplete by construction. Reset, measurement, and dynamic-control semantics do not share the same exact unitary argument. The frozen v2 benchmark uses synthetic circuits, durations, and errors, with five seeds and 36 held-out cases; these support paired claims within that benchmark, not general state-of-the-art ranking. Learned-policy fallback preserves termination but weakens conclusions about autonomous policy quality. The v2 direction-dependent cost replay corrected saved cost fields without rerunning the original 792 routes.

The supplementary study has 30 single runs and one SABRE seed, with no uncertainty intervals; its aggregate depends on a selected 15/15 mix of hub and chain-like/dense circuits. Per-run figures were transcribed from dashboard screenshots and require comparison with saved result files before submission. The displayed validation panel established Weighted's pass status, not independent validation of both baseline outputs. The displayed depth should also be checked against the manuscript's pre- versus post-directed-lowering definitions. The claimed causes of Weighted's failures are hypotheses: comparing CP-SAT-optimal $J$ to the returned layout on runs 2, 7, 24, 27–30 would distinguish local-search trapping from a weak objective, while random logical relabelling would test the identity-placement explanation. No claim of hardware-calibrated fidelity, reproducible speed advantage, or statistical

TABLE V: Complete 30-run supplementary comparison. $D_0$: source depth; SW: inserted SWAPs; $D$: routed depth; ms: single dashboard compile-time observation. These runs are separate from frozen benchmark v2.

| | | | | | QWeave Weighted | | | QWeave Basic | | | Qiskit SABRE | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| # | Circuit | $n$ | $D_0$ | Topology | SW | $D$ | ms | SW | $D$ | ms | SW | $D$ | ms |
| 1 | GHZ | 4 | 4 | Linear path | 0 | 4 | 31.2 | 0 | 4 | 4.4 | 0 | 4 | 37.0 |
| 2 | GHZ | 6 | 6 | Linear path | 4 | 10 | 28.1 | 0 | 6 | 12.6 | 0 | 6 | 24.1 |
| 3 | GHZ | 6 | 6 | Ring | 0 | 6 | 21.4 | 0 | 6 | 7.7 | 0 | 6 | 21.4 |
| 4 | GHZ | 6 | 6 | Grid 2x3 | 0 | 6 | 15.2 | 2 | 8 | 14.2 | 2 | 7 | 55.1 |
| 5 | GHZ | 6 | 6 | Chorded ring | 0 | 6 | 18.4 | 0 | 6 | 9.9 | 0 | 6 | 22.4 |
| 6 | GHZ | 4 | 4 | Star | 1 | 5 | 13.6 | 2 | 6 | 11.0 | 1 | 5 | 22.3 |
| 7 | GHZ | 4 | 4 | IBM Manila | 1 | 5 | 13.4 | 0 | 4 | 13.0 | 0 | 4 | 36.2 |
| 8 | GHZ | 4 | 4 | Grid 2x2 | 0 | 4 | 7.2 | 1 | 5 | 6.8 | 1 | 4 | 22.9 |
| 9 | Star | 4 | 3 | Linear path | 1 | 4 | 13.6 | 2 | 5 | 6.3 | 2 | 5 | 32.7 |
| 10 | Star | 4 | 3 | Linear path | 1 | 4 | 10.2 | 2 | 5 | 31.9 | 2 | 5 | 64.0 |
| 11 | Star | 4 | 3 | Ring | 1 | 4 | 10.5 | 2 | 5 | 8.6 | 1 | 4 | 24.6 |
| 12 | Star | 4 | 3 | Grid 2x2 | 1 | 4 | 15.9 | 1 | 4 | 12.6 | 1 | 3 | 36.5 |
| 13 | Star | 4 | 3 | Star | 0 | 3 | 13.3 | 0 | 3 | 8.5 | 0 | 3 | 25.6 |
| 14 | Star | 5 | 4 | Linear path | 3 | 7 | 14.3 | 3 | 7 | 10.3 | 3 | 7 | 27.9 |
| 15 | Star | 5 | 4 | Ring | 3 | 7 | 17.6 | 3 | 7 | 16.0 | 3 | 7 | 30.0 |
| 16 | Star | 5 | 4 | Grid 2x3 | 1 | 5 | 18.6 | 3 | 7 | 12.4 | 2 | 6 | 28.5 |
| 17 | Star | 5 | 4 | Chorded ring | 1 | 5 | 9.5 | 3 | 7 | 13.4 | 1 | 5 | 20.0 |
| 18 | Star | 5 | 4 | Star | 0 | 4 | 9.0 | 0 | 4 | 9.6 | 0 | 4 | 24.9 |
| 19 | Star | 5 | 4 | IBM Manila | 1 | 5 | 15.4 | 3 | 7 | 9.3 | 3 | 6 | 24.6 |
| 20 | BV | 6 | 5 | Linear path | 0 | 5 | 37.2 | 5 | 8 | 24.7 | 3 | 7 | 31.8 |
| 21 | BV | 6 | 5 | Ring | 0 | 5 | 43.4 | 2 | 5 | 12.8 | 2 | 6 | 34.0 |
| 22 | BV | 6 | 5 | Grid 2x3 | 0 | 5 | 33.0 | 2 | 5 | 16.9 | 1 | 6 | 32.5 |
| 23 | BV | 6 | 5 | Chorded ring | 0 | 5 | 129.2 | 2 | 6 | 61.5 | 1 | 6 | 51.8 |
| 24 | HEA | 4 | 18 | Linear path | 10 | 28 | 51.2 | 9 | 27 | 21.5 | 9 | 29 | 29.4 |
| 25 | HEA | 4 | 18 | Ring | 0 | 18 | 25.2 | 0 | 18 | 19.2 | 0 | 18 | 27.9 |
| 26 | HEA | 4 | 18 | Grid 2x2 | 0 | 18 | 30.3 | 1 | 19 | 30.7 | 1 | 18 | 37.1 |
| 27 | HEA | 4 | 18 | Star | 11 | 29 | 68.0 | 11 | 29 | 55.8 | 7 | 29 | 45.0 |
| 28 | HEA | 4 | 18 | IBM Manila | 10 | 28 | 75.8 | 9 | 27 | 53.2 | 9 | 29 | 57.5 |
| 29 | QFT | 4 | 24 | Linear path | 6 | 32 | 35.1 | 7 | 31 | 22.8 | 5 | 29 | 33.5 |
| 30 | QFT | 4 | 24 | Ring | 3 | 26 | 28.5 | 2 | 25 | 20.5 | 2 | 26 | 29.6 |

TABLE VI: Supplementary study: SWAP totals by circuit structure and topology category (Weighted / Basic / SABRE). Each cell aggregates a different number of runs.

| Circuit group | Short-distance: grids, chorded ring | Intermediate: ring, star | Sparse trees: linear, Manila |
|---|---|---|---|
| Hub-structured (BV, star graph) | **3 / 11 / 6** ($n = 5$) | **4 / 7 / 6** ($n = 5$) | **6 / 15 / 13** ($n = 5$) |
| Chain-like/dense (GHZ, HEA, QFT) | **0 / 4 / 4** ($n = 4$) | 15 / 15 / **10** ($n = 5$) | 31 / 25 / **23** ($n = 6$) |

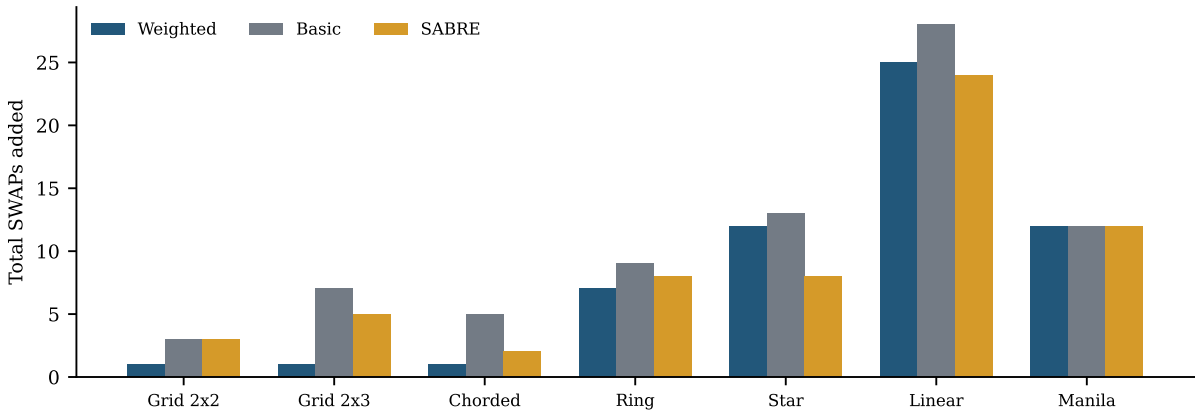



Fig. 5: Supplementary 30-run SWAP totals grouped by topology. Groups pool different physical sizes and circuit families; these totals are descriptive, not controlled topology-effect estimates.

significance follows from the supplementary timings.

## IX. Conclusion

QWeave combines a deterministic initial-mapping and routing reference, a small exact mapping oracle, trace-backed semantic validation, fixed-route scheduling, directed-gate lowering, and guarded learned routing. Its formal arguments specify the invariants, feasibility, and termination properties supported by the declared contract. In frozen benchmark v2, PPO training substantially improves an untrained graph policy, but neither the learned router nor message passing improves on weighted deterministic routing. The additional 30-run study finds 59

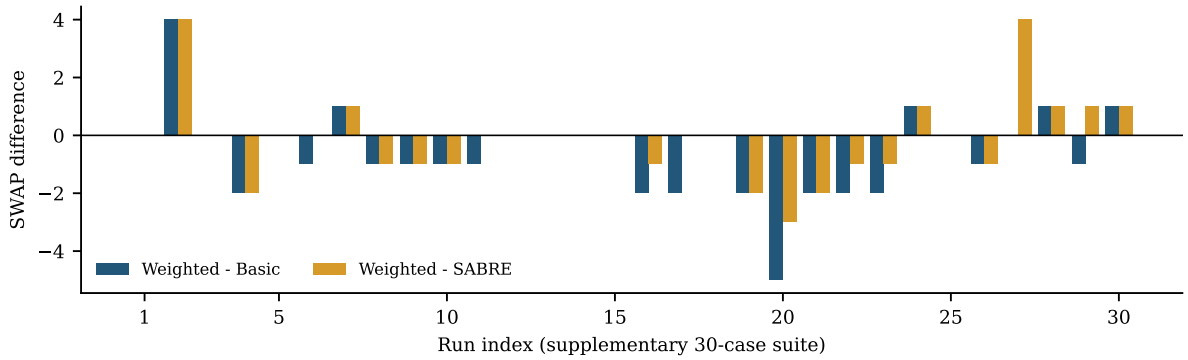



Fig. 6: Supplementary per-run differences in inserted SWAPs. Negative values mean Weighted inserted fewer SWAPs than the named reference; positive values show counterexamples.

Weighted SWAPs versus 77 Basic and 62 SABRE, with total routed depths of 297, 306, and 300, respectively. Crucially, this aggregate advantage is driven by hub-structured circuits and is offset by regressions on chain-like or dense circuits, especially on sparse hardware. The supported contribution is a correctness-first compiler and a workload-sensitive empirical characterization, not universal routing superiority. A promising *untested* direction is to combine weighted initial placement with order-aware look-ahead or critical-path-aware routing and evaluate it under a new frozen, preregistered protocol.

## REFERENCES


[1] IBM Quantum, "Transpiler stages," IBM Quantum Documen- tation, 2026.

[2] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in Proc. ASPLOS, 2019.

[3] M. G. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins, "Using reinforcement learning to perform qubit routing in quantum compilers," arXiv:2007.15957, 2020.

[4] A. Sinha, U. Azad, and H. Singh, "Qubit routing using graph neural network aided Monte Carlo tree search," in Proc. AAAI, 2022.

[5] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018.

[6] K. Bharti et al., "Noisy intermediate-scale quantum algo- rithms," Rev. Mod. Phys., vol. 94, no. 1, p. 015004, 2022.

[7] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th anniversary ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.

[8] A. Javadi-Abhari et al., "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[9] N. Khammassi et al., "OpenQL: A portable quantum program- ming framework for quantum accelerators," ACM J. Emerg. Technol. Comput. Syst., vol. 18, no. 1, pp. 1-24, 2021.

[10] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan, "tlket>: A retargetable compiler for NISQ devices," Quantum Sci. Technol., vol. 6, no. 1, p. 014003, 2020.

[11] M. Bandic, S. Feld, and C. G. Almudever, "Full-stack quantum computing systems in the NISQ era: Algorithm-driven and hardware-aware compilation techniques," in Proc. DATE, pp. 1-6, 2022.

[12] M. Y. Siraichi, V. F. dos Santos, S. Collange, and F. M. Q. Pereira, "Qubit allocation," in Proc. CGO, pp. 113-125, 2018.

[13] A. Zulehner, A. Paler, and R. Wille, "An efficient methodology for mapping quantum circuits to the IBM QX architectures," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 38, no. 7, pp. 1226-1236, 2019.

[14] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah, "On the qubit routing problem," in Proc. TQC, LIPIcs, vol. 135, pp. 5:1-5:32, 2019.

[15] A. M. Childs, E. Schoute, and C. M. Unsal, "Circuit transfor- mations for quantum architectures," in Proc. TQC, LIPIcs, vol. 135, pp. 3:1-3:24, 2019.

[16] R. Wille, O. Keszocze, M. Walter, P. Rohrs, A. Chattopadhyay, and R. Drechsler, "Look-ahead schemes for nearest neighbor optimization of 1D and 2D quantum circuits," in Proc. ASP- DAC, pp. 292-297, 2016.

[17] A. Lye, R. Wille, and R. Drechsler, "Determining the minimal number of SWAP gates for multi-dimensional nearest neighbor quantum circuits," in Proc. ASP-DAC, pp. 178-183, 2015.

[18] S. Li, X. Zhou, and Y. Feng, "Qubit mapping based on subgraph isomorphism and filtered depth-limited search," IEEE Trans. Comput., vol. 70, no. 11, pp. 1777-1788, 2021.

[19] T. Itoko, R. Raymond, T. Imamichi, and A. Matsuo, "Optimiza- tion of quantum circuit mapping using gate transformation and commutation," Integration, vol. 70, pp. 43-50, 2020.

[20] B. Tan and J. Cong, "Optimal qubit mapping with simultaneous gate absorption," arXiv:2109.06445, 2021.

[21] M. A. Steinberg, S. Feld, C. G. Almudever, M. Marthaler, and J.-M. Reiner, "Topological-graph dependencies and scaling properties of a heuristic qubit-assignment algorithm," IEEE Trans. Quantum Eng., vol. 3, pp. 1-14, 2022.

[22] IBM Quantum, "SabreLayout API reference," IBM Quantum Documen- tation, 2026.

[23] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in Proc. ASPLOS, pp. 1015-1029, 2019.

[24] S. S. Tannu and M. K. Qureshi, "Not all qubits are created equal: A case for variability-aware policies for NISQ-era quan- tum computers," in Proc. ASPLOS, pp. 987-999, 2019.

[25] S. Nishio, Y. Pan, T. Satoh, H. Amano, and R. Van Meter, "Extracting success from IBM's 20-qubit machines using error- aware compilation," ACM J. Emerg. Technol. Comput. Syst., vol. 16, no. 3, pp. 1-25, 2020.

[26] L. Lao, H. van Someren, I. Ashraf, and C. G. Almudever, "Timing and resource-aware mapping of quantum circuits to superconducting processors," IEEE Trans. Comput.-Aided De- sign Integr. Circuits Syst., vol. 41, no. 2, pp. 359-371, 2022.

[27] G. G. Guerreschi and J. Park, "Two-step approach to scheduling quantum circuits," Quantum Sci. Technol., vol. 3, no. 4, p. 045003, 2018.

[28] P. Murali et al., "Full-stack, real-system quantum computer studies: Architectural comparisons and design insights," in Proc. ISCA, pp. 527-540, 2019.

[29] S. Hillmich, A. Zulehner, and R. Wille, "Exploiting quantum teleportation in quantum circuit mapping," in Proc. ASP-DAC, pp. 792-797, 2021.

[30] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in Proc. Computing Frontiers, pp. 98-107, 2020.

[31] M. Bandic et al., "Mapping quantum circuits to modular architectures with QUBO," arXiv:2305.06687, 2023.

[32] L. Lao and D. E. Browne, "2QAN: A quantum com- piler for 2-local qubit Hamiltonian simulation algorithms," arXiv:2108.02099, 2021.

[33] M. Bandic, H. Zarein, E. Alarcon, and C. G. Almudever, "On structured design space exploration for mapping of quantum algorithms," in Proc. DCIS, pp. 1-6, 2020.

[34] S. Herbert and A. Sengupta, "Using reinforcement learning to find efficient qubit routing policies for deployment in near-term quantum computers," arXiv:1812.11619, 2018.

[35] G. Acampora and R. Schiattarella, "Deep neural networks for quantum circuit mapping," Neural Comput. Appl., vol. 33, pp. 13723-13743, 2021.

[36] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. ICLR, 2017.

[37] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in Proc. ICML, PMLR, vol. 70, pp. 1263-1272, 2017.

[38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[39] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," arXiv:1506.02438, 2015.

[40] G. Nannicini, L. S. Bishop, O. Gunluk, and P. Jurcevic, "Op- timal qubit assignment and routing via integer programming," ACM Trans. Quantum Comput., vol. 4, no. 1, 2022.

[41] D. Venturelli, M. Do, E. Rieffel, and J. Frank, "Compiling quan- tum circuits to realistic hardware architectures using temporal planners," Quantum Sci. Technol., vol. 3, no. 2, p. 025004, 2018.

[42] D. Venturelli et al., "Quantum circuit compilation: An emerging application for automated reasoning," in Proc. SPARK, 2019.

[43] M. Bandic, C. G. Almudever, and S. Feld, "Interaction graph- based characterization of quantum benchmarks for improving quantum circuit mapping techniques," Quantum Mach. Intell., vol. 5, p. 40, 2023.

[44] A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation, and J. M. Gambetta, "Validating quantum computers using randomized model circuits," Phys. Rev. A, vol. 100, p. 032328, 2019.

[45] R. Blume-Kohout and K. C. Young, "A volumetric framework for quantum computer benchmarks," Quantum, vol. 4, p. 362, 2020.

[46] D. Mills, S. Sivarajah, T. L. Scholten, and R. Duncan, "Application-motivated, holistic benchmarking of a full quan- tum computing stack," Quantum, vol. 5, p. 415, 2021.

[47] T. Tomesh et al., "SupermarQ: A scalable quantum benchmark suite," in Proc. HPCA, pp. 587-603, 2022.

[48] A. Li and S. Krishnamoorthy, "QASMBench: A low-level QASM benchmark suite for NISQ evaluation and simulation," arXiv:2005.13018, 2020.

[49] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approx- imate optimization algorithm," arXiv:1411.4028, 2014.

[50] D. Coppersmith, "An approximate Fourier transform useful in quantum factoring," arXiv:quant-ph/0201067, 2002.

[51] T. Häner, D. S. Steiger, K. Svore, and M. Troyer, "A software methodology for compiling quantum programs," Quantum Sci. Technol., vol. 3, no. 2, p. 020501, 2018. PDF: https://arxiv.org/ pdf/1604.01401.

[52] A. JavadiAbhari et al., "ScaffCC: Scalable compilation and analysis of quantum programs," Parallel Comput., vol. 45, pp. 2–17, 2015. PDF: https://arxiv.org/pdf/1507.01902.

[53] R. S. Smith, M. J. Curtis, and W. J. Zeng, "A practical quantum instruction set architecture," arXiv:1608.03355, 2016. PDF: https://arxiv.org/pdf/1608.03355.

[54] X. Fu et al., "eQASM: An executable quantum instruction set architecture," in Proc. MICRO, pp. 224–237, 2019. PDF: https: //arxiv.org/pdf/1808.02449.

[55] H. Deng, Y. Zhang, and Q. Li, "CODAR: A contex- tual duration-aware qubit mapping for various NISQ de- vices," arXiv:2002.10915, 2020. PDF: https://arxiv.org/pdf/ 2002.10915.

[56] Y. Ding, P. Gokhale, S. F. Lin, R. Rines, T. Propson, and F. T. Chong, "Systematic crosstalk mitigation for superconducting qubits via frequency-aware compilation," in Proc. MICRO, 2020. PDF: https://arxiv.org/pdf/2008.09503.

[57] W. Finigan, M. Cubeddu, T. Lively, J. Flick, and P. Narang, "Qubit allocation for noisy intermediate-scale quantum com- puters," arXiv:1810.08291, 2018. PDF: https://arxiv.org/pdf/ 1810.08291.

[58] A. Zulehner and R. Wille, "Compiling SU(4) quantum circuits to IBM QX architectures," in Proc. ASP-DAC, pp. 185–190, 2019. PDF: https://arxiv.org/pdf/1808.05661.

[59] R. Wille and L. Burgholzer, "MQT QMAP: Efficient quantum circuit mapping," in Proc. ISPD, pp. 198–204, 2023. PDF: https: //arxiv.org/pdf/2301.11935.

[60] F. Hua et al., "A synergistic compilation workflow for tackling crosstalk in quantum machines," arXiv:2207.05751, 2022. PDF: https://arxiv.org/pdf/2207.05751.

[61] M. A. Nielsen, "A simple formula for the average gate fidelity of a quantum dynamical operation," Phys. Lett. A, vol. 303, no. 4, pp. 249–252, 2002. PDF: https://arxiv.org/pdf/quant-ph/ 0205035.

[62] A. Gilchrist, N. K. Langford, and M. A. Nielsen, "Distance measures to compare real and ideal quantum processes," Phys. Rev. A, vol. 71, p. 062310, 2005. PDF: https://arxiv.org/pdf/ quant-ph/0408063.

[63] E. Magesan, J. M. Gambetta, and J. Emerson, "Characterizing quantum gates via randomized benchmarking," Phys. Rev. A, vol. 85, p. 042311, 2012. PDF: https://arxiv.org/pdf/1109.6887.

[64] J. J. Wallman and J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling," Phys. Rev. A, vol. 94, p. 052325, 2016. PDF: https://arxiv.org/pdf/1512. 01098.

[65] A. Erhard et al., "Characterizing large-scale quantum comput- ers via cycle benchmarking," Nature Commun., vol. 10, p. 5347, 2019. PDF: https://arxiv.org/pdf/1902.08543.

[66] T. J. Proctor, A. Carignan-Dugas, K. Rudinger, E. Nielsen, R. Blume-Kohout, and K. Young, "Direct randomized bench- marking for multi-qubit devices," Phys. Rev. Lett., vol. 123, p. 030503, 2019. PDF: https://arxiv.org/pdf/1807.07975.

[67] A. Hashim et al., "Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor," Phys. Rev. X, vol. 11, p. 041039, 2021. PDF: https://arxiv. org/pdf/2010.00215.

[68] T. Lubinski et al., "Application-oriented performance bench- marks for quantum computing," IEEE Trans. Quantum Eng., vol. 4, pp. 1–32, 2023. PDF: https://arxiv.org/pdf/2110.03137.

[69] S. Martiel, T. Ayral, and C. Allouche, "Benchmarking quantum co-processors in an application-centric, hardware-agnostic and scalable way," IEEE Trans. Quantum Eng., vol. 2, pp. 1–11, 2021. PDF: https://arxiv.org/pdf/2102.12973.

[70] S. Resch and U. R. Karpuzcu, "Benchmarking quantum com- puters and the impact of quantum noise," ACM Comput. Surv., vol. 54, no. 7, 2022. PDF: https://arxiv.org/pdf/1912.00546.

[71] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proc. AAAI, vol. 32, 2018. PDF: https://arxiv.org/pdf/1709.06560.

[72] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Proc. NeurIPS, vol. 34, pp. 29304– 29320, 2021. PDF: https://arxiv.org/pdf/2108.13264.

[73] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive represen- tation learning on large graphs," in Proc. NeurIPS, vol. 30, pp. 1024–1034, 2017. PDF: https://arxiv.org/pdf/1706.02216.

[74] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in Proc. ICLR, 2018. PDF: https://arxiv.org/pdf/1710.10903.

[75] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015. PDF: https://arxiv.org/ pdf/1412.6980.